

Lecture 37: Tabular Foundation Models

A model that never saw your data, and never saw anyone's

Predict first

Two competitors on a tabular classification problem with about a thousand rows.

A. An ensemble of the strongest baselines you know from ch4 - boosting and friends - **tuned for four hours.**

B. A transformer that has **never seen a single real dataset**, given **2.8 seconds** and no tuning at all.

Which one wins?

Predict first

Two competitors on a tabular classification problem with about a thousand rows.

A. An ensemble of the strongest baselines you know from ch4 - boosting and friends - **tuned for four hours.**

B. A transformer that has **never seen a single real dataset**, given **2.8 seconds** and no tuning at all.

Which one wins?

B. And not narrowly.

The claim, and its boundary, in the same breath

Hollmann et al., *Accurate predictions on small data with a tabular foundation model*, **Nature** 637, 9 January 2025.

"TabPFN... outperforms all previous methods on datasets with **up to 10,000 samples** by a wide margin, using substantially less training time. **In 2.8 s, TabPFN outperforms an ensemble of the strongest baselines tuned for 4 h** in a classification setting."

Note what the sentence does *not* say. It is not "beats trees". It is "**up to 10,000 samples**".

That number is the whole lecture. Everything after this frame is either why it works, or why the number is there.

A distinction to keep straight. 10,000 is where it *wins*. The same paper reports *running* on up to 50 million cells - say 5 million rows by 10 features - on one H100 GPU. **Capacity and superiority are different claims.**

What chapter 4 told you

The boosting deck closed on this, in a takeaway box:

"Trees + ensembles dominate tabular data."

That was true. It is still true across most of the space. And the Nature paper opens by saying the same thing about the previous twenty years:

"gradient-boosted decision trees have dominated tabular data for the past 20 years"

So this lecture is not a refutation. It is a **boundary**. We are going to find the edge of a claim the course has already made, and say precisely where it is.

Outline

The inversion

The architecture

The landscape

What is it actually doing?

Where it stops winning

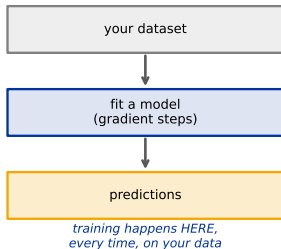
The inversion

Stop fitting the model to the data.

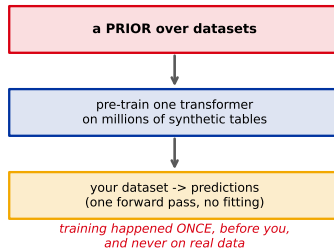
Fitted to what?

Every model in chapters 1 to 8 is fitted to *your* table. A **Prior-Data Fitted Network (PFN)** is fitted to a *prior over tables*, once, by somebody else.

Ordinary supervised learning



A prior-data fitted network



The Nature paper's own phrase: "*a learning algorithm that is itself learned across millions of synthetic datasets.*"

The recipe, in four steps

1. **Write down a prior** - an explicit generative process over datasets.
2. **Sample millions of synthetic datasets** from it.
3. **Train one transformer** to take $(\mathbf{X}_{\text{train}}, \mathbf{y}_{\text{train}}, \mathbf{X}_{\text{test}})$ and output a predictive distribution for $\mathbf{y}_{\text{test}}$.
4. **Ship it.** It is never trained again.

The object being learned is not a function $\hat{f}$ for one dataset. It is **an algorithm** that maps any dataset to a prediction. Step 3 is the only training that ever happens, and it happens before you exist.

At inference there is no fitting at all

You do not call `fit` in the usual sense. Your **entire training table** goes into the model's context, the test rows follow it, and the answer comes out of **one forward pass**.

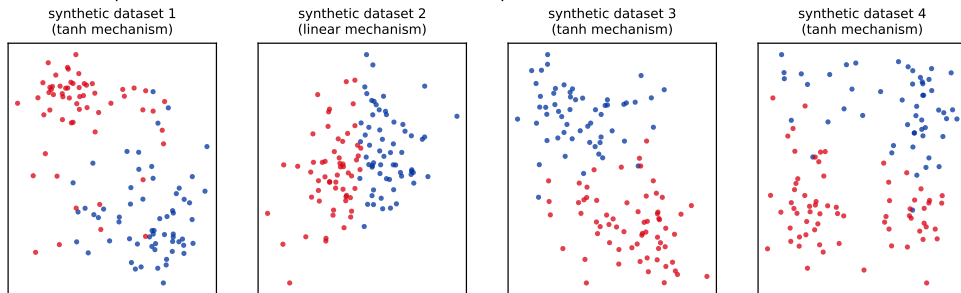
This is exactly **in-context learning**, which you met for language models - the same mechanism that lets an LLM pick up a pattern from examples in the prompt without any weight update. Here the "prompt" is a spreadsheet.

Consequence: there are no epochs, no learning rate, no early stopping, and nothing to tune. There is also no fitted object to save - reuse means feeding the training table through again.

The prior, part 1: what is actually in it

Pre-training data is **synthetic**. Datasets are sampled from a prior built on **structural causal models (SCMs)** and **Bayesian neural networks (BNNs)**: draw a random causal graph, draw random mechanisms, push noise through, read off features and a label.

Draws from a prior over structural causal models - TabPFN pre-trains on millions of these, and on no real tables



Schematic draws from a small prior of this kind, generated for this slide - not TabPFN's actual prior, which is far richer.

The prior, part 2: the prior *is* the inductive bias

This is the frame to hold on to. Everything later in the lecture is a consequence of it.

A tree's inductive bias is axis-aligned splits. A linear model's is additivity. A convolutional network's is locality.

A PFN's inductive bias is a probability distribution that somebody wrote down in code.

Three later frames are just this fact reappearing:

- ▶ shift needs its **own prior** to be handled;
- ▶ the prior's bias **does not wash out** with more data;
- ▶ benchmarks that resemble the prior **flatter the model**.

Change the prior and you have changed the model, without touching the architecture.

"No training" is not "no cost"

A misconception worth killing early.

The pre-training **happened**. It was expensive. It ran once, on somebody else's hardware, on millions of generated datasets.

What changed is **who pays, and when**. The cost moved from your machine at your deadline to their machine, once, in advance.

That is the same trade as any pretrained model in this course - ch6's transfer learning, ch9's language models. The novelty is not free computation. It is that here the pre-training data was **invented** rather than collected.

The architecture

A table is not a sentence, and pretending otherwise is unaffordable.

One representation per cell

The obvious thing is to flatten the table into a sequence of tokens and use an ordinary transformer. TabPFN does not. From the paper:

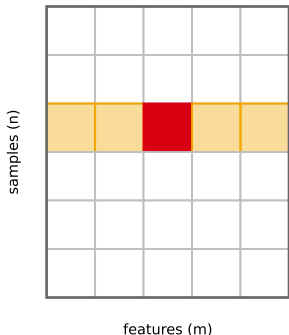
"an architecture that assigns a separate representation to **each cell** in the table... a **two-way attention** mechanism, with each cell attending to the other features in its row (that is, its sample) and then attending to the same feature across its column (that is, all other samples)."

So the state is indexed by **(row, column)**, not by position in a 1-D stream, and there are **two** distinct attention operations per layer rather than one.

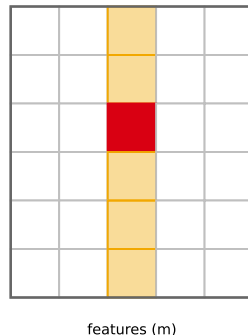
Two-way attention

Every cell gets its own representation, and attends twice per layer

1. Attention over FEATURES
a cell attends across its own row



2. Attention over SAMPLES
the same cell attends down its own column



Then an MLP sublayer, with residual connections and normalisation, exactly as in ch9. Nothing about the transformer block itself is new here. What is new is **what attends to what**.

Predict first

Take your test table and **permute the columns** - same data, different order. Then permute the rows.

Do the predictions change?

Predict first

Take your test table and **permute the columns** - same data, different order. Then permute the rows.

Do the predictions change?

No. And not because the model learned not to care. The paper: this design **"enables the architecture to be invariant to the order of both samples and features."**

The invariance is architectural. Same idea as a convolutional network's translation equivariance in ch6 - build the symmetry into the wiring instead of hoping data teaches it.

A row-order-dependent model would be absurd for a spreadsheet. A sequence model would have to learn its way out of that. This one cannot express it in the first place.

Worked numbers: why the factorisation is not just tidy

Suppose n rows and m features, so $n \cdot m$ cells.

Naive: one sequence of all $n \cdot m$ cells. Attention is quadratic in sequence length, so $(n \cdot m)^2$ pairs.

Two-way: across a row is m wide, down a column is n tall. The paper states the result as

$$\mathcal{O}(n^2 + m^2) \text{ compute,} \quad \mathcal{O}(n \cdot m) \text{ memory.}$$

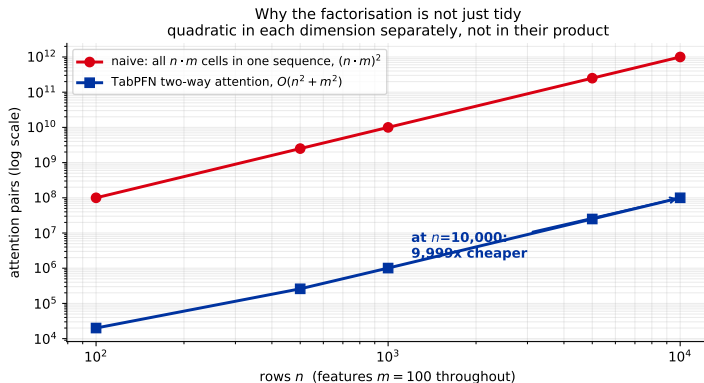
Quadratic in each dimension **separately**, not in the product.

At $n = 10,000$ and $m = 100$:

naive	10^{12}
two-way	10^8
ratio	$\approx 10,000\times$

The architecture is not only about invariance. It is what makes putting a whole dataset in a context window **affordable at all**.

The cost, plotted



Memory is linear, $\mathcal{O}(n \cdot m)$, and the paper reports under 1,000 bytes per cell after optimisation. **The binding constraint is rows**, which is exactly where the 10,000 in the headline came from - and why the next section is mostly about rows.

The landscape

One idea, several models, and a licence you should read.

The line, and the neighbours

When	What	Why it matters
2021	Transformers Can Do Bayesian Inference	the founding idea, before tables
2022	TabPFN v1	proof of concept, classification only
2025	TabPFN v2 (Nature)	the result this lecture is about
2025	TabPFN-2.5	50,000 samples, 2,000 features
2026	TabPFN-3	current line

It is not a monopoly. The alternatives differ on the two axes that actually matter - **where the pre-training data comes from** and **how many rows fit**:

- ▶ **TabICL** (Tabular In-Context Learning) and TabICLv2 - built for larger tables, and open.
- ▶ **TabDPT** - pre-trains on **real** data instead of a purely synthetic prior.
- ▶ **LimiX** - a further entrant, 2026.

The idea also extends: time-series forecasting, distribution shift, wide tables, synthetic data generation, multitask.

Using it

The whole interface, for the model that beat four hours of tuning in 2.8 seconds:

```
from tabpfn import TabPFNClassifier
clf = TabPFNClassifier()
clf.fit(X_train, y_train)
p = clf.predict_proba(X_test)
```

`fit` here does **not** train anything. It stores your table so it can be fed into the context at predict time. The scikit-learn interface is a courtesy, and it hides the fact that all the work happens in `predict`.

What is it actually doing?

Nobody agrees, including the people who built it.

Why does it improve on tables bigger than anything it pre-trained on?

That question has no agreed answer, and the disagreement is in print.

The Bayesian reading. The network approximates the **posterior predictive** under the prior. Generalisation in context is Bayesian updating. *Müller, Hollmann & Hutter (2024)* - the TabPFN authors.

The frequentist reading. A PFN is a **pre-tuned but untrained** predictor. No posterior needed - and this version explains improvement on tables larger than any seen in pre-training. *Nagler (ICML 2023)*.

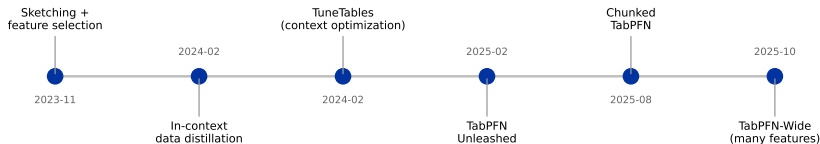
And a third result unsettles both. For causal effect estimation, **the prior induces a bias that more data does not asymptotically wash away** (Melnychuk et al., 2026). Consistency is the property you were taught to expect from an estimator. Here it fails - because, as two frames ago, **the prior is the model**.

Where it stops winning

The honest half. It is most of the lecture's value.

Six papers exist only to make it take more rows

Six papers whose whole purpose is fitting more rows in the context
that is the shape of the method, not a frontier



When a method needs a six-paper literature to move one number, that number is not a frontier being pushed - it is **the shape of the method**. Compare: nobody writes papers to let a random forest read more rows.

Distribution shift needed its own model

Real tables drift. Prices change, populations change, the sensor gets recalibrated.

Drift-Resilient TabPFN (2024) handles temporal shift by putting shift **into the prior**: pre-train on synthetic datasets that already drift.

Elegant - and it tells you plainly that **the base model does not handle it**. If a phenomenon is not in the prior, it is not in the model.

This is the cleanest illustration of the frame you were told to hold on to. Capability is bought by editing the generative process, not by collecting more of your data.

The benchmark problem

A model pre-trained on a prior over structural causal models will do well on benchmark datasets that **look like draws from that prior**.

Realistic Evaluation of TabPFN v2 in Open Environments (2025) asks what happens when the quiet assumptions of a benchmark suite stop holding
- clean features, no shift, a closed label set.

That is not a criticism unique to this model. It is the question you should ask of any reported win.

You have the tools for this already. This is `27_stat_how_to_lie` applied to a **benchmark culture** rather than to a single experiment.

Feature engineering: chapter 6 already measured this

A tempting overclaim is that this kills feature engineering. Chapter 6 got there first, with numbers, and was more careful:

Feature engineering was worth +81.5 **MAE to Ridge** and -4.3 **to a forest**.
"Feature engineering is not dead - it moved."

A capable model already absorbed most of what feature engineering used to buy - ch6 measured it *hurting* the forest. A tabular foundation model pushes further in the same direction.

It does not touch the part ch6 said still pays: **joins, domain knowledge, and the features no model can derive because the information is not in the table**. Research in 2026 (*TabPrep*) is actively arguing the gap is still there.

The licence

This one changes what you are allowed to build, so it is not a footnote.

TabPFN-2.5 weights ship under a licence permitting research and internal evaluation, and **excluding commercial or production use** - explicitly including the model's **outputs**.

"Outputs" is broad. Generating a feature with it, or labelling data with it, produces a covered output.

Which makes **TabICLv2** being open a real engineering decision, not a footnote: slightly different model, entirely different set of things you may ship.

"Beats gradient boosting" and "cannot go in your product" belong on the same slide.

Recap

1. A prior-data fitted network is trained on a **prior over datasets**, not on your dataset. Your table is context, not training data.
2. **The prior is the inductive bias.** Everything else follows: drift needs its own prior, benchmark flattery, bias that will not wash out.
3. Two-way attention gives **order invariance by construction** and $\mathcal{O}(n^2 + m^2)$ instead of $\mathcal{O}((nm)^2)$ - about $10,000\times$ at $n=10,000$.
4. It genuinely beats four hours of tuning in 2.8 seconds - **up to about 10,000 rows**.
5. Nobody agrees on what it is doing, including its authors.

What to do with this.

Chapter 4's rule stands: **on tabular data, reach for boosting** - above roughly ten thousand rows, under shift, and anywhere you have to ship commercially.

Below that, on a small clean table, try the forward pass first. It costs seconds, and chapter 4's four hours of tuning is the thing you are saving.

The transferable idea: **if you can write down where your problems come from, you can learn the algorithm instead of the answer.**